maps

Release 1.0.0

AE,SB,BB

May 20, 2026

Contents:

CHAPTER

ONE

CLFUNCS MODULE

clfuncs.**correlate_fast**(*GV*, *ni*, *corr*)

Helper function (**numba jit accelerated**) which performs the **Cross TGE** estimator.

Parameters

- **GV** (*np.ndarray*) – Convolved gridded visibility array (3D array) $\mathcal{V}_{cg}(\nu_n)^{XX/YY}$. **GV** structure is (Polarization, Grid points, Frequency).
- **ni** (*np.ndarray*) – 1D array containing the info about which grid point falls into which annular bin. Must contain values in between $0, \text{Nbin} - 1$.
- **corr** (*np.ndarray*) – 3D array of shape (Nbin, NC, NC).

Returns

This function does not return anything.

Return type

None

clfuncs.**correlate**(*GV*, *ni*, *Nbin*)

Given the gridded visibility data, it performs correlation using **Cross TGE** estimator which is defined as follows :

$$\text{corr}(\nu_a, \nu_b) = \mathcal{R}e\left[\mathcal{V}_{cg}^{\text{RR}}(\nu_a)\mathcal{V}_{cg}^{*\text{LL}}(\nu_b) + \mathcal{V}_{cg}^{\text{LL}}(\nu_a)\mathcal{V}_{cg}^{*\text{RR}}(\nu_b)\right]$$

Parameters

- **GV** (*np.ndarray*) – Convolved gridded visibility array (3D array) $\mathcal{V}_{cg}(\nu_n)^{XX/YY}$. **GV** structure is (Polarization, Grid points, Frequency).
- **ni** (*np.ndarray*) – 1D array containing the info about which grid point falls into which annular bin. Must contain values in between $0, \text{Nbin} - 1$.
- **Nbin** (*int*) – No of annular bins the whole uv plane has been divided into.

Returns

corr – 3D array of shape (Nbin, NC, NC).

Return type

np.ndarray

Examples

```
>>> import numpy as np
>>> import clfuncs as cfunc
Imported clfuncs, Import Time : 06/05/26 | 07:42:26 PM
>>> GV = np.load('GV_7320_pool.npy')
>>> print(f'GV.shape   : {GV.shape}') # GV shape [nstokes,ni,NC] with nstokes = 2
GV.shape   : (2, 46438, 768)
>>> # Load binning info
>>> BIN  = np.load('BIN_7200.npz')
>>> # No of annular bins the whole uv plane has been divided into
>>> Nbin = int(BIN['Nbin'])
>>> # Contains info about which grid point fall into which bin
>>> ni   = BIN['ni']
>>> print(f'No of Bins : {Nbin}')
No of Bins : 20
>>> # perform correlation
>>> corr = cfunc.correlate(GV, ni, Nbin)
Channels   : 768
Time Taken : 11.381 seconds.
>>> # corr shape (Nbin, NC, NC)
>>> print(f'corr.shape :{corr.shape}') # GV shape [nstokes,ni,NC] with nstokes = 2
corr.shape :(20, 768, 768)
```

clfuncs.cl_dnu_nubar(*maps*)

Given the *Multi-Angular Power Specrtrum (MAPS)* $C_\ell(\nu_a, \nu_b)$ array, it computes $C_\ell(\bar{\nu}, \Delta\nu)$.

Parameters

maps (*np.ndarray*) – *MAPS* $C_\ell(\nu_a, \nu_b)$ array. Must be shape of (Nbin, NC, NC).

Returns

maps1 – *MAPS* $C_\ell(\bar{\nu}, \Delta\nu)$ array having shape (Nbin, NC, 2NC -1).

Return type

np.ndarray

Examples

```
>>> import numpy as np
>>> import clfuncs as cfunc
Imported clfuncs, Import Time : 07/05/26 | 10:37:02 AM
>>> maps = np.load('cl_maps_7200_noscf.npy')
>>> # shape (Nbin, NC, NC)
>>> print(f'maps.shape   : {maps.shape}')
maps.shape   : (20, 668, 668)
>>> maps1 = cfunc.cl_dnu_nubar(maps)
>>> # shape (Nbin, NC, 2*NC -1)
>>> print(f'maps1.shape   : {maps1.shape}')
maps1.shape   : (20, 668, 1335)
```

clfuncs.cl_dnu(*maps1*)

Given the $C_\ell(\bar{\nu}, \Delta\nu)$ array, it computes $C_\ell(\Delta\nu)$ by averaging over $\bar{\nu}$ axis.

Parameters
`maps1` (*np.ndarray*) – $C_\ell(\bar{\nu}, \Delta\nu)$ array. Must be shape of (Nbin, NC, 2NC -1).

Returns
$C_\ell(\Delta\nu)$ array having shape (Nbin, NC).

Return type
np.ndarray

Examples

```
>>> maps2 = cfunc.cl_dnu(maps1)
>>> # shape (Nbin, NC)
>>> print(f'maps2.shape   : {maps2.shape}')
maps2.shape   : (20, 668)
```

`clfuncs.cl_dnu_nua_nub`(*maps*)

Given the *Multi-Angular Power Specrtrum (MAPS)* $C_\ell(\nu_a, \nu_b)$ array, it computes $C_\ell(\Delta\nu)$. Internally it computes $C_\ell(\bar{\nu}, \Delta\nu)$ first and then using this, it computes $C_\ell(\Delta\nu)$.

Parameters
`maps` (*np.ndarray*) – *MAPS* $C_\ell(\nu_a, \nu_b)$ array. Must be shape of (Nbin, NC, NC).

Returns
$C_\ell(\Delta\nu)$ array having shape (Nbin, NC).

Return type
np.ndarray

Examples

```
>>> maps2 = cfunc.cl_dnu_nua_nub(maps)
>>> # shape (Nbin, NC)
>>> print(f'maps2.shape   : {maps2.shape}')
maps2.shape   : (20, 668)
```

CHAPTER

TWO

CORRELATE MODULE

`correlate.correlate_fast(`*GV*, *ni*, *corr*`)`

Helper function (**numba jit accelerated**) which performs the **Cross TGE** estimator.

Parameters

- `GV` (*np.ndarray*) – Convolved gridded visibility array (3D array) $\mathcal{V}_{cg}(\nu_n)^{XX/YY}$. **GV** structure is (Polarization, Grid points, Frequency).
- `ni` (*np.ndarray*) – 1D array containing the info about which grid point falls into which annular bin. Must contain values in between $0, \text{Nbin} - 1$.
- `corr` (*np.ndarray*) – 2D array of shape (Nbin, NC).

Returns

This function does not return anything.

Return type

None

`correlate.correlate(`*GV*, *ni*, *Nbin*`)`

Given the gridded visibility data, it performs correlation using **Cross TGE** estimator which is defined as follows :

$$\text{corr}(\nu_a, \nu_b) = \mathcal{R}e\left[\mathcal{V}_{cg}^{\text{RR}}(\nu_a)\mathcal{V}_{cg}^{*\text{LL}}(\nu_b) + \mathcal{V}_{cg}^{\text{LL}}(\nu_a)\mathcal{V}_{cg}^{*\text{RR}}(\nu_b)\right]$$

Then it collapses the frequency axis to get $\text{corr}(\Delta\nu)$.

Parameters

- `GV` (*np.ndarray*) – Convolved gridded visibility array (3D array) $\mathcal{V}_{cg}(\nu_n)^{XX/YY}$. **GV** structure is (Polarization, Grid points, Frequency).
- `ni` (*np.ndarray*) – 1D array containing the info about which grid point falls into which annular bin. Must contain values in between $0, \text{Nbin} - 1$.
- `Nbin` (*int*) – No of annular bins the whole uv plane has been divided into.

Returns

corr – 2D array of shape (Nbin, NC).

Return type

np.ndarray

Examples

```
>>> import numpy as np
>>> import correlate
Imported correlate, Import Time : 20/05/26 | 06:03:15 PM
>>> # Load binning info
>>> BIN  = np.load('/home/baijayanta/MY_Documented_Code/tge/bin_info_7200.npz')
>>> # No of annular bins the whole uv plane has been divided into
>>> Nbin = int(BIN['Nbin'])
>>> # Contains info about which grid point fall into which bin
>>> ni   = BIN['ni']
>>>
>>> GV = np.load('/home/baijayanta/CFFI_C/GV_7200_pool.npy')
>>> # GV shape [nstokes,ni,NC] with nstokes = 2
>>> print(f'{GV.shape   = }')
GV.shape   = (2, 46438, 768)
>>>
>>> print(f'No of Bins = {Nbin}')
No of Bins = 20
>>>
>>> # perform correlation
>>> corr = correlate.correlate(GV, ni, Nbin)
Channels   : 768
Time Taken : 7.400 seconds.
>>> print(f'{corr.shape   = }')
corr.shape   = (20, 768)
```